

What Does a VPN Actually Hide?

A hands-on AirVPN verification using WireGuard, packet captures, DNS tests, kill-switch failure testing, IPv6 bypass attempts, and provider-trust analysis

Kyle Pakhet / Fragmented Sentinel

The question

A VPN clearly encrypts traffic between a device and a VPN server. But what does that actually hide from an ISP, what can still leak, and what trust simply moves to the VPN provider? This paper tests those questions rather than accepting either marketing claims or blanket skepticism.

Independent technical test - 26 September 2026 | Version 1.0 | Published 27 September 2026

DOI: <https://doi.org/10.5281/zenodo.22986653>

Source inspiration: NetworkChuck, “VPN myths / what a VPN actually does.” [1]

Abstract

This paper documents a practical verification of an AirVPN connection from an Ubuntu virtual machine using WireGuard. The investigation began with a simple question raised by a NetworkChuck video: does a VPN meaningfully hide destination information from an ISP, or does it merely move the visibility to the VPN provider? The answer, based on direct packet capture and deliberate bypass testing, is both.

The VPN successfully concealed DNS queries and destination connections from the VM's external interface. IPv4 and IPv6 were both routed through the tunnel. Hard-coded public DNS remained inside the VPN. Attempts to force traffic through the physical VMware interface failed after a persistent nftables kill switch was repaired. The most important finding was that the original kill switch had silently failed at boot, demonstrating why configuration files and “connected” indicators are not substitutes for testing.

The server-side trust question is harder. AirVPN publicly states that it does not monitor or log online activity and that it uses physical/bare-metal servers. TorrentFreak records similar provider claims, while RTINGS reports that AirVPN has not published an independent third-party audit of its no-logging policy or infrastructure. [2][4][5] The practical conclusion is therefore narrower than either “VPNs are useless” or “VPNs make you private”: the tested VPN materially reduced ISP-visible destination metadata, but the remaining trust was concentrated in AirVPN and the surrounding hosting/network environment.

1. Why test this at all?

The investigation started after watching NetworkChuck test the idea that HTTPS already makes a personal VPN unnecessary. His demonstration showed that HTTPS protects content but can still leave useful destination metadata visible to a local observer; he then showed that once traffic was

encapsulated in WireGuard, the local capture mainly saw the VPN tunnel. The video also made the more important point that a VPN shifts trust from the ISP to the VPN provider. [1]

Rather than debating those claims in the abstract, I used an existing AirVPN WireGuard VM and asked a series of progressively harder questions:

- Does normal IPv4 traffic actually use the VPN?
- Does IPv6 use the VPN, or can it escape separately?
- Where do DNS queries go, and what is visible inside versus outside the tunnel?
- What happens if the VPN drops?
- Can an application bypass the tunnel by binding directly to the physical interface?
- Can an application use a hard-coded DNS resolver without exposing the home connection?
- Does the protection survive a reboot?
- What infrastructure sits at the other end of the tunnel, and what does that imply for trust?

2. Test setup and evidence standard

The test system was an Ubuntu 24.04 virtual machine under VMware. Its ordinary network interface was connected to VMware NAT, while a separate `airvpn` interface was created by `wg-quick`. The AirVPN peer was a New York endpoint using UDP port 1637, with `AllowedIPs = 0.0.0.0/0, ::/0`, so the intended design was a full-tunnel IPv4/IPv6 VPN.

The evidence standard was deliberately conservative. Results were classified as one of three types:

1. Directly observed: routing-table output, nftables state, packet captures, DNS responses, forced-interface tests, reboot journals, and public IP tests.
2. Provider-asserted: statements made by AirVPN about logging, server type, privacy practices, or infrastructure.
3. Externally reported: findings or provider responses published by sources such as RTINGS, TorrentFreak, Akamai, and ARIN.

Why this matters

A successful packet capture can prove that traffic did or did not leave a specific interface. It cannot prove that a VPN provider never logs traffic after it arrives at the provider. That boundary is central to the conclusions in this paper.

3. The most valuable finding: the kill switch was broken

The first major result was not reassuring: the persistent firewall that was supposed to prevent non-VPN traffic had failed during boot. The original nftables configuration referenced the `airvpn` interface using rules that required the interface to already exist. At boot, WireGuard had not yet created it, so nftables failed to load. The ruleset also allowed an obsolete AirVPN endpoint rather than the endpoint currently configured in WireGuard.

When WireGuard was deliberately stopped, the route to the Internet fell back to the VMware NAT gateway and the VM regained ordinary Internet access outside the VPN. That is the exact failure a kill switch is supposed to prevent.

```

VPN stopped
-> route fell back to VMware NAT
-> ordinary Internet access continued
-> non-VPN public IP became visible

```

The firewall was repaired by using interface-name matching (`iifname` / `oifname`) so the rules could be loaded before the WireGuard interface existed, and by allowing direct traffic only to the configured AirVPN WireGuard endpoints. The default output policy remained `drop`, while normal Internet traffic was permitted only through the `airvpn` interface.

After the repair, stopping WireGuard caused Internet access to fail closed. Restarting WireGuard restored connectivity.

4. DNS: visible inside the tunnel, absent outside it

The VM used systemd-resolved. AirVPN installed `10.128.0.1` and an IPv6 resolver on the VPN link, with the route-only domain `~.`, which tells systemd-resolved to prefer that link for general DNS resolution.

The key experiment was to capture port 53 traffic on both sides of the tunnel while resolving several domains.

```

Outside capture:
tcpdump -pni ens33 'port 53'
-> 0 packets captured

Inside capture:
tcpdump -pni airvpn 'port 53'
-> A/AAAA queries for airvpn.org, kernel.org, eff.org and wikipedia.org were visible

```

The DNS packets themselves were ordinary DNS on port 53 inside the tunnel. That does not constitute a DNS leak: WireGuard encrypted the entire inner packet before it reached the external interface. This distinction is important. Encryption does not require every inner protocol to be independently encrypted if the entire packet is already protected by the VPN tunnel.

5. What the ISP-side capture could still see

A broader packet capture was then taken on the external interface while HTTPS requests were made to multiple websites. Instead of separate connections to those destinations, the outside interface showed UDP traffic between the VM and the AirVPN WireGuard endpoint. The destination DNS names and destination IP connections were not exposed as separate flows on that interface.

Observable outside the tunnel	Not directly observable outside the tunnel
AirVPN endpoint IP	DNS queries carried inside WireGuard
UDP protocol / tunnel port	Destination IP flows carried inside WireGuard
Packet timing and direction	TLS SNI belonging to inner HTTPS sessions
Packet sizes and traffic volume	HTTPS content, URLs, passwords, messages
Session duration / traffic bursts	Application-level traffic as separate Internet flows

This is the first major qualification to the phrase “a VPN hides your metadata.” It hides a large amount of destination-related metadata from the local/ISP side, but it does not make the connection invisible. Timing, packet size, direction, duration, and the VPN endpoint remain observable. A sufficiently capable adversary with visibility on both sides of a VPN may attempt traffic correlation without decrypting WireGuard.

6. Deliberate bypass tests

Normal routing tests are useful, but they do not answer what happens when software deliberately tries to avoid the expected path. The VM was therefore tested as if an application were misconfigured or actively attempting to bypass the VPN.

Test	Method	Result
Force HTTPS through physical interface	<code>`curl -interface ens33 ...`</code>	Timed out; no Internet packets captured on <code>`ens33`</code>
Force IPv4 ICMP through physical interface	<code>`ping -I ens33 1.1.1.1`</code>	100% packet loss
Force local VMware DNS	<code>`dig @192.168.107.2 example.com`</code>	Timed out; no port-53 packets left <code>`ens33`</code>
Hard-coded public DNS	<code>`dig @1.1.1.1 example.com`</code>	Succeeded, but packet capture showed it traversed <code>`airvpn`</code>
Force IPv6 through physical interface	<code>`curl -6 --interface ens33 ...`</code>	Failed
Force IPv6 ICMP through physical interface	<code>`ping -6 -I ens33 ...`</code>	Network unreachable
IPv6 through AirVPN	<code>`curl -6 --interface airvpn ...`</code>	Succeeded with an AirVPN/tzulo IPv6 exit

The hard-coded ``1.1.1.1`` result is particularly useful. It demonstrates the difference between a DNS-policy bypass and a VPN leak. An application can ignore AirVPN’s DNS resolver and use Cloudflare directly, but the query still travelled through WireGuard. Cloudflare would therefore see the VPN exit address rather than the user’s home address.

7. Reboot and boot-order verification

Because the original firewall failure was a boot-order problem, a real reboot was necessary. After reboot, both `nftables` and ``wg-quick@airvpn`` were active, IPv4 and IPv6 public-IP tests returned AirVPN/tzulo addresses, and the live `nftables` ruleset contained both the persistent kill-switch table and WireGuard’s temporary IPv4/IPv6 helper tables.

The boot journal showed `nftables` completing before ``wg-quick@airvpn``. More importantly, ``systemctl show nftables`` reported that `nftables` is ordered before ``network-pre.target`` with ``DefaultDependencies=no``. In other words, the firewall is intentionally placed before normal networking rather than merely happening to win a race on one boot.

```

system boot
-> persistent nftables policy loads
-> network-pre.target
-> normal networking
-> WireGuard interface and routes
-> Internet becomes available through AirVPN

```

8. Identifying the AirVPN infrastructure

The WireGuard endpoint resolved to a tzulo customer hostname. ARIN/IP allocation data and independent network data identify the surrounding range as tzulo / AS11878. [8] The public VPN exit used a different address in the same network. This provided direct evidence that the tested server used separate entry and exit IPv4 addresses.

The DNS path yielded another useful result. Akamai's `whoami.ds.akahelp.net` diagnostic reports the public source address of the recursive resolver that contacted Akamai and can also report EDNS Client Subnet (ECS) data when present. [6] When queried through AirVPN's internal resolver, the result showed a resolver egress address in the same AirVPN/tzulo IPv6 network and no ECS field was observed.

Finally, CHAOS-class DNS queries returned `version.bind = "AirVPN DNS"` and `id.server = "unurgunite.airservers.org"`. The resolver therefore identified the AirVPN node as Unurgunite while suppressing a normal DNS software version string. The DNS resolver egress and the web IPv6 exit were different addresses within the same `/64`, which is consistent with closely colocated services but does not prove whether they run on the same physical machine.

9. Where the trust moves

At this point the client-side question was largely answered: the local/ISP side saw the WireGuard tunnel, not the separate DNS and destination flows inside it. But encryption between the client and the VPN server does not make the provider blind.

Once traffic reaches the VPN server, the WireGuard layer must be processed so that packets can be forwarded. For ordinary HTTPS, end-to-end TLS still protects the actual page contents, passwords, and messages from the VPN provider. However, the VPN provider is technically positioned to observe or correlate information such as the incoming VPN connection, destination IP addresses, timing, traffic volume, and DNS queries when its resolver is used.

The trust shift

Before the VPN, the ISP has direct visibility into much of the destination-level network metadata. With the VPN, the ISP mainly sees the encrypted tunnel to AirVPN. AirVPN then occupies the privileged network position at the other end. The visibility is not destroyed; much of it is relocated.

10. What AirVPN says - and what outside evidence adds

AirVPN publicly advertises “No monitoring nor logging of your online activities,” “high performance physical servers,” and account creation without mandatory personal information or even a real email address. [2] AirVPN staff have also stated that the service does not keep logs that can identify a customer or VPN client IP and does not monitor or inspect client traffic. [3]

TorrentFreak’s 2026 VPN privacy survey records AirVPN saying it does not retain data that would allow an IP address and timestamp to be matched to a current or former user, uses no monitoring tools for connection enforcement, and operates real bare-metal servers rather than VPS or virtual locations. The same response acknowledges an important limitation: transparent external wiretapping around server lines cannot necessarily be detected or prevented from inside the VPN server. [4] These are provider statements collected by TorrentFreak, not an independent technical audit.

RTINGS provides a useful independent counterweight. Its current AirVPN review reports no ordinary IP or DNS leaks while connected, but reports that AirVPN’s tested kill switch leaked traffic during a reboot. More importantly for the trust question, RTINGS says AirVPN has not published independent third-party audits of its no-logging policy or security infrastructure. [5]

The reboot-leak result does not contradict this paper’s final client-side result because this test system used a separate, persistent system-level nftables policy rather than relying solely on the provider client’s default kill-switch behaviour. The custom firewall was explicitly tested through tunnel loss, direct-interface binding, DNS bypass attempts, IPv6 attempts, and reboot.

There is also behavioural evidence about AirVPN’s privacy posture. In 2024, AirVPN stopped accepting Italian residents after Italy’s Piracy Shield framework extended rapid blocking obligations to VPN/DNS intermediaries. TorrentFreak reported that AirVPN chose withdrawal rather than implementing those blocking requirements. [7] That behaviour may inform a trust assessment, but it is not proof of no logging.

11. Evidence matrix

Claim	Evidence type	Confidence / scope
IPv4 and IPv6 use the VPN	Directly verified	Strong
DNS is contained in the VPN	Direct packet capture	Strong
Physical-interface bypass is blocked	Direct adversarial test	Strong
VPN failure is fail-closed after repair	Direct failure test	Strong
Protection survives reboot	Direct reboot + journal test	Strong
Separate entry/exit addressing on tested node	Direct public-IP / endpoint comparison	Strong for this node
AirVPN resolver does not send ECS in tested query	Akamai diagnostic	Strong for this query
AirVPN does not retain identifying traffic logs	Provider claim; external survey repeats claim	Not independently proven here
AirVPN uses bare-metal servers	Provider claim; TorrentFreak records claim	Not independently proven here

Claim	Evidence type	Confidence / scope
All AirVPN nodes behave like this one	Not tested	Unknown

12. What this paper does not prove

This was a point-in-time test of one Ubuntu VM, one WireGuard configuration, and one AirVPN server path. It was not an audit of AirVPN's production environment. The following remain outside the scope of the evidence collected here:

- Whether AirVPN's production systems ever retain identifying traffic data despite the published policy.
- Whether every AirVPN server uses identical logging, DNS, networking, and hardening configuration.
- Whether the tested physical server is owned by AirVPN, rented directly from tzulo, colocated, or supplied through another dedicated arrangement.
- What access the hosting/network provider has at the server, hypervisor (if any), switching, or upstream network layers.
- Whether a capable adversary observing both sides of the VPN could successfully correlate traffic.
- Whether browser cookies, logins, fingerprinting, application telemetry, or other non-network identifiers can still identify the user. A VPN is not an anonymity system by itself.

13. Conclusion

The original question was whether a personal VPN meaningfully hides useful information from an ISP or merely moves the problem somewhere else. The testing supports both propositions.

On the client side, the VPN did exactly what a full-tunnel VPN is supposed to do after the kill switch was repaired: IPv4, IPv6, DNS, hard-coded public DNS, and ordinary HTTPS traffic remained inside WireGuard; forced attempts to use the physical interface failed; tunnel loss failed closed; and the protection survived reboot. Packet captures showed a clear difference between the inner interface, where DNS and destination traffic were visible, and the outer interface, where the observable Internet flow was the WireGuard relationship with the AirVPN endpoint.

That is meaningful privacy improvement. It is not invisibility. The ISP can still see that a VPN is being used and can observe timing, direction, volume, packet sizes, and session duration. And at the other end of the tunnel, AirVPN becomes the privileged network intermediary.

AirVPN's observable behaviour in this test was consistent with several of its privacy-oriented claims: separate entry/exit addressing, a local AirVPN DNS service, no ECS observed in the diagnostic query, IPv4/IPv6 support, and a network path that kept tested traffic inside the tunnel. AirVPN also publicly states that it does not log or inspect client activity. [2][3][4]

The largest unresolved evidence gap is independent verification of that server-side no-logging claim. RTINGS reports that AirVPN has not published a third-party privacy/no-logs or infrastructure audit. [5] That does not mean the claim is false; it means the strongest claim in the trust chain remains self-attested rather than independently demonstrated.

Bottom line

A VPN does not eliminate trust. It relocates and concentrates it. Direct testing can reduce blind trust on the client side; transparent architecture and independent audits can reduce it on the provider side. In this test, AirVPN materially reduced ISP-visible destination metadata, while the remaining uncertainty was concentrated primarily in AirVPN's server-side logging practices and the surrounding hosting/network environment.

14. Practical takeaway: verify your own VPN

The most transferable lesson was not specific to AirVPN. A VPN can appear healthy while a kill switch or leak-prevention control is broken. A small verification routine provides much stronger assurance than a green “connected” icon.

- Confirm IPv4 and IPv6 route selection.
- Capture DNS on the external interface while generating fresh lookups.
- Capture traffic on the VPN interface to understand what is being encapsulated.
- Stop the VPN and confirm that Internet traffic fails closed.
- Bind test traffic directly to the physical interface and verify it cannot escape.
- Test a hard-coded public DNS resolver and determine whether it stays inside the VPN.
- Reboot and verify firewall/VPN startup ordering.
- Separate provider marketing claims from independently testable behaviour and independent audit evidence.

Appendix A - Representative test commands

```
# Route selection
ip route get 1.1.1.1
ip -6 route get 2606:4700:4700::1111

# DNS outside and inside the tunnel
sudo tcpdump -pni ens33 'port 53'
sudo tcpdump -pni airvpn 'port 53'

# Force physical-interface bypass attempts
curl --interface ens33 --connect-timeout 5 https://ifconfig.me
ping -I ens33 -c 3 1.1.1.1

# Local DNS bypass
dig @192.168.107.2 example.com +time=2 +tries=1

# Hard-coded public DNS
dig @1.1.1.1 example.com +time=2 +tries=1

# IPv6 bypass
curl -6 --interface ens33 --connect-timeout 5 https://ifconfig.me

# AirVPN DNS identity / resolver diagnostics
dig +short TXT whoami.ds.akahelp.net @10.128.0.1
dig @10.128.0.1 CH TXT version.bind
dig @10.128.0.1 CH TXT id.server

# Boot verification
journalctl -b -u nftables -u wg-quick@airvpn --no-pager
systemctl show nftables -p Before -p After -p Wants -p DefaultDependencies
```

References

- [1] NetworkChuck - source video that prompted the investigation
<https://www.youtube.com/watch?v=axfNxZ1R6C4>
- [2] AirVPN - official service claims and features
<https://airvpn.org/>
- [3] AirVPN staff - “Logging on VPN servers?”
<https://airvpn.org/forums/topic/9717-logging-on-vpn-servers/?comment=11242&do=findComment>
- [4] TorrentFreak - “Which VPN Providers Take Privacy Seriously in 2026?”
<https://torrentfreak.com/best-vpn-anonymous-no-logging/>
- [5] RTINGS - AirVPN review (current test bench / privacy and kill-switch findings)
<https://www.rtings.com/vpn/reviews/airvpn/airvpn>
- [6] Akamai - resolver WhoAmI / ECS diagnostic documentation
<https://www.akamai.com/blog/developers/introducing-new-whoami-tool-dns-resolver-information>
- [7] TorrentFreak - AirVPN withdrawal from Italy over Piracy Shield requirements (2024)
<https://torrentfreak.com/airvpn-stops-serving-italians-due-to-piracy-shield-blocking-requirements-240206/>

[8] ARIN - Registration Data Access Protocol (RDAP)

<https://www.arin.net/resources/registry/whois/rdap/>

[9] AirVPN - example server announcement describing no logs and separate entry/exit IP addresses

<https://airvpn.org/forums/topic/74049-new-10-gbits-server-available-se-and-replacement/>

[10] WireGuard - official quick-start documentation

<https://www.wireguard.com/quickstart/>

Publication note

This document intentionally omits the tester's home public IP address, account information, private cryptographic material, and other unnecessary identifying information. It is published under the creator name Kyle Pakhet / Fragmented Sentinel. Public AirVPN infrastructure addresses are included only where they materially explain the network path. The test reflects the observed configuration on 26 September 2026 and should not be read as a permanent certification of AirVPN or of every server in its network.

Suggested citation: Kyle Pakhet / Fragmented Sentinel. (2026). What Does a VPN Actually Hide?

Independent Verification of AirVPN, WireGuard, DNS and Kill-Switch Behaviour (Version 1.0). Zenodo.

<https://doi.org/10.5281/zenodo.22986653>